Table of Contents

1. Pre-requisites 2

2. Purpose with the Problem scenario 2

3. Introduction 2

4. Detailed Explanation 3

4.1. Content 3

4.1.1. Relating real world scenario using technical Concepts 3

4.1.2. Theory 7

4.1.3. Pros & Cons 14

4.1.4. Syntax 15

4.2. Code Snippets 15

4.3. Real Time Usage 15

4.4. Check Yourself 15

5. Error Zone - commonly made mistakes 16

6. Sample code 16

7. Additional Information (Optional) 16

8. Best Practices 16

9. Summary or Take Away Points 16

10. Related Topics: 17

11. References 17

1. Pre-requisites

Should have the basic understanding on Classes and Objects , Object Oriented programming, Constructors, Inheritance.

2. Purpose with the Problem scenario

Consider a scenario where an organization has to track the details of its employees. There can be two types of employees – Regular and Contract. All the employees will have some common attributes like EmployeeId, Name, Designation, ContactNumber. The Regular Employees will have Monthly Basic whereas Contract Employees will have Hourly Rate and Number of Hours worked in a month. The details of employees depend on the type of employee. How can we ensure that only the respective details are displayed?

There are two approaches – one is to declare separate methods to display different types of employees

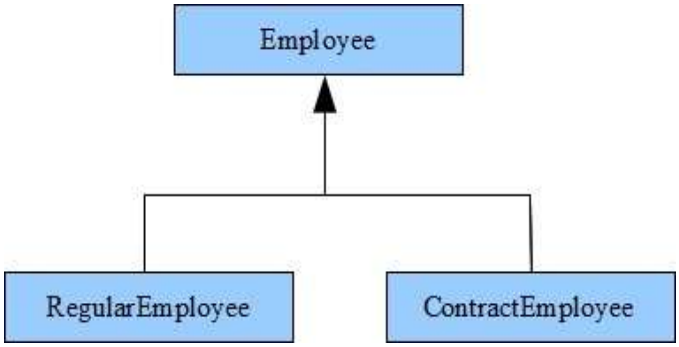
The second approach is to have a class Employee having a display method. Create subclasses Regular and Contract Employee classes by extending/inheriting the Employee class. Redefine the display method in both the classes. At run time, depending on the object on which the display method is invoked, the respective behavior would be executed.

The second approach has the advantages : re-usability and readability.

3. Introduction

Polymorphism means existence in multiple forms. In the context of Object oriented programming languages Polymorphism means - Subclasses of a class can define their own unique behaviors and yet share some of the same functionality of the parent class.

In the scenario mentioned in above section, the classes Regular Employee and Contract Employee are inheriting/extending the class Employee. There is a method display() in Employee class that prints the Employee details. In classes RegularEmployee and ContractEmployee, we redefine the display() method to show the additional attributes present in each of these classes ie these classes are giving their own behavior for display() method and same time sharing the attributes or some other methods defined in the parent class.



The concept of polymorphism can be achieved in two ways:

- By adding new methods in sub class having same name as the methods in super class but with different parameter list (Method Overloading).
- By re-defining the methods present in super class (called Method Overriding). The methods will have same signature as the methods in super class.

4. Detailed Explanation

4.1. Content

4.1.1. Relating real world scenario using technical Concepts

For the Organization scenario mentioned above, the class Employee will be as follows:

```
class Employee {
        int empId;
        String empName;
        String designation;

        Employee(int empId, String empName, String designation){
                this.empId=empId;
                this.empName=empName;
                this.designation=designation;
        }
        public void display(){
                        System.out.println("EmpId:"+empId+ "EmpName:"+
                                        empName+" Designation:"+designation);
        }
}
```

Both Regular and Contract employees have the

properties mentioned in Employee above and additionally some specific properties. In this case, we will reuse them by extending the Employee class and then redefine the display method.

```
class RegularEmployee extends Employee {
        int basic;
        RegularEmployee(int empId, String empName, String designation,int basic){
                super(empId, empName, designation);
                this.basic=basic;
        }
        public void display(){
                System.out.println("Displaying RegularEmployee Details");
                super.display();
                System.out.println("Basic:"+basic);
        }
    }

class ContractEmployee extends Employee {
        int hourlyRate;
        double numHours;
        ContractEmployee(int empId, String empName, String designation,int hourlyRate,
                                                double numHours){
                super(empId, empName, designation);
                this.hourlyRate=hourlyRate;
                this.numHours=numHours;
        }

        public void display(){
                System.out.println("Displaying ContractEmployee Details");
                super.display();
                System.out.println("HourlyRate:"+hourlyRate+ "NumHours:"+numHours);
        }
}
```

Now the class RegularEmployee will have the attribute basic as well as the attributes inherited from Employee ie. empId, empName, designation. The method display() present in Employee class is redefined in RegularEmployee. Similar is the case with ContractEmployee.

```
class EmployeeDetails {
        public static void main(String[] args){
                Employee e = new Employee(1111,"ABC","Developer");
                e.display();//line 4
                System.out.println();

                RegularEmployee re1 = new RegularEmployee(2222, "XYZ",
                        "Developer", 1000);
                re1.display();  //line 8
                System.out.println();

                ContractEmployee re2 = new ContractEmployee(3333,"MNO",
                        "Developer", 100, 200);
                re2.display();  //line 12
        }
}
```

Let us consider the following code snippet where we try to invoke display method on Regular and Contract employees:

Look at lines 4,8 and 12 where we are invoking the display method on the objects of Employee, RegularEmployee and ContractEmployee classes respectively. The output will be as follows:

For e.display(),

EmpId:1111 EmpName:ABC Designation:Developer

For re1.display(),

Displaying RegularEmployee Details

EmpId:2222 EmpName:XYZ Designation:Developer

Basic:1000

For re2.display(),

Displaying ContractEmployee Details

EmpId:3333 EmpName:MNO Designation:Developer

HourlyRate:100NumHours:200.0

When we redefine a method in a sub class and whenever we invoke this method on subclass object, the version of the method in subclass will be invoked. If the method is not

redefined, the version of the method in base class will be invoked. Thus same method call will exhibit different behaviors depending on whether it is redefined in subclass or not.

4.1.2. Theory

I. Method Overloading:

Method overloading is the concept of having different methods having same name but different parameters. The parameters can differ in:

- Number of parameters
- Type of parameters
- Order of parameters

Suppose that you have a class that can use calligraphy to draw various types of data (strings, integers, and so on) and that contains a method for drawing each data type. It is cumbersome to use a new name for each method—for example, drawString, drawInteger, drawFloat, and so on. In the Java programming language, you can use the same name for all the drawing methods but pass a different argument list to each method. Thus, the data drawing class might declare four methods named draw, each of which has a different parameter list.

```
public class DataArtist {
    ...
    public void draw(String s) {        //1st method
        ...
    }
    public void draw(int i) {           //2nd method
        ...
    }
    public void draw(double f) {        //3rd method
        ...
    }
    public void draw(int i, double f) { //4th method
        ...
    }
    public void draw(double x, int y) { //5th method
        ...
    }
}
```

Overloaded methods are differentiated by the number , type and order of the arguments passed into the method. In the code sample above,

- 1st, 2nd and 3rd methods are differentiated by the type of arguments

- 4th method is distinguished from 1st, 2nd and 3rd by the number of arguments
- 4th and 5th methods are distinguished by the order of arguments.

You cannot declare more than one method with the same name and the same number , type and orderof arguments, because the compiler cannot distinguish them.

The compiler does not consider return type when differentiating methods, so you cannot declare two methods with the same signature even if they have a different return type.

Following are the rules for method overloading:

- The overloaded methods should have same name.
- The argument list should differ in number of parameters or type of parameters or order of parameters.
- The return type is of no significance in overloading.
- The access specifier is of no significance in overloading.
- Static methods can be overloaded.
- Final methods can be overloaded.
- Abstract methods can be overloaded.
- The exceptions thrown by the methods are of no significance.
- Constructors can be overloaded.

Method overloading can be done both within the same class as well as in an inheritance hierarchy. The same rules as mentioned above will be applicable to inheritance hierarchy as well, except that constructors cannot be overloaded across classes.

Note: Overloaded methods should be used sparingly, as they can make code much less readable. Overloading is also called as compile time polymorphism.

II. Method Overriding:

Method overriding is the process where a method defined in super class is redefined in sub class. The signature of the method in subclass should be same as the signature of the method in super class.

Following are the rules for method overriding:

- The name of the method in super class and sub class is same.
- The argument list of the method in sub class should exactly be the same as that of the overridden method.
- The return type should be the same or a subtype of the return type declared in the original overridden method in the super class. This subtype is called a co-variant return type.
- The access level cannot be more restrictive than the overridden method's access level. For example: if the super class method is declared public then the overriding method in the sub class cannot be either private or public. However the access level can be less restrictive than the overridden method's access level.
- Instance methods can be overridden only if they are inherited by the subclass.

- A method declared final cannot be overridden, but it can be overloaded.
- A method declared static cannot be overridden but can be re-declared. This is called method hiding.
- A method declared abstract can be overridden.
- A method declared private in super class cannot be inherited and hence cannot be overridden. We can declare a method with same signature in sub class, however it is not a overriding. It is as if we added a new method in sub class.
- A subclass within the same package as the instance's superclass can override any superclass method that is not declared private or final.
- A subclass in a different package can only override the non-final methods declared public or protected.
- An overriding method can throw any unchecked exceptions, regardless of whether the overridden method throws exceptions or not. However the overridden method should not throw checked exceptions that are new or broader than the ones declared by the overridden method. The overriding method can throw narrower or fewer exceptions than the overridden method.
- Constructors cannot be overridden

Following table summarizes the use of access specifiers wrt. method overriding:

Access Specifier for method in Base Class	**Applicable access specifiers for overridden method in sub class**
public	public
protected	protected, public
no specifier	no specifier, protected, public
private	private methods cannot be inherited and hence no overriding. We can re-declare the same method in sub class with any access specifier

III. **Method Hiding:**

➢ If a subclass defines a static method with the same signature as a static method in the superclass, then the method in the subclass hides the one in the superclass.

➢ The distinction between hiding a static method and overriding an instance method has important implications:

•The version of the overridden instance method that gets invoked is the one in the subclass.

•The version of the hidden static method that gets invoked depends on whether it is invoked from the superclass or the subclass.

Consider an example that contains two classes. The first is Animal, which contains one instance method and one static method:

```
public class Animal {
        public static void testClassMethod() {
        System.out.println("The static method in Animal");
        }
        public void testInstanceMethod() {
        System.out.println("The instance method in Animal");
        }
}
```

The second class, a subclass of Animal, is called Cat:

```
public class Cat extends Animal {
   public static void testClassMethod() {
      System.out.println("The static method in Cat");
   }
   public void testInstanceMethod() {
      System.out.println("The instance method in Cat");
   }
   public static void main(String[] args) {
      Cat myCat = new Cat();
      Animal myAnimal = myCat;
      Animal.testClassMethod();
      myAnimal.testInstanceMethod();
   }
}
```

The Cat class overrides the instance method in Animal and hides the static method in Animal. The main method in this class creates an instance of Cat and invokes testClassMethod() on the class and testInstanceMethod() on the instance.

The output from this program is as follows:

The static method in Animal

The instance method in Cat

As mentioned, the version of the hidden static method that gets invoked is the one in the superclass, and the version of the overridden instance method that gets invoked is the one in the subclass.

- The following table summarizes what happens when you define a method with the same signature as a method in a superclass.

Defining a Method with the Same Signature as a Superclass's Method

	Superclass Instance Method	**Superclass Static Method**
Subclass Instance Method	Overrides	Generates a compile-time error
Subclass Static Method	Generates a compile-time error	Hides

IV. **Types of Polymorphism:**

- Depending on the time (Compilation or Execution of a program) at which the resolution of a method call will be decided, Polymorphism can be classified as :
  - Static Polymorphism and
  - Dynamic Polymorphism
- If the method binding between method call and method definition happens at compile time, it is called Static Polymorphism. It is also called Compile time polymorphism or static binding or early binding.
- If the method binding between method call and method definition happens at runtime, it is called Dynamic Polymorphism. It is also called Run time binding or dynamic binding. Here Java compiler does not understand which method is called at compilation time. Only JVM decides which method is called at run-time this is called dynamic method dispatch
- Java supports Static Polymorphism using Method Overloading, Method hiding using static, private methods. For final methods also, Static Polymorphism is used as they cannot be overridden in sub class.
- Java supports Dynamic Polymorphism with method overriding using instance methods.
- **Example for Static Polymorphism:**

```
public class StaticBindingTest {
        public static void main(String args[])  {
          Collection c = new HashSet();
          StaticBindingTest et = new StaticBindingTest();
          et.sort(c);
        }
        //overloaded method takes Collection argument
        public Collection sort(Collection c){
           System.out.println("Inside Collection sort method");
           return c;
        }
       //another overloaded method that takes HashSet argument
        public Collection sort(HashSet hs){
           System.out.println("Inside HashSet sort method");
           return hs;
        }
}
```

In the above example the binding of method to et.sort(c) would be done at compilation time based on the argument(s) to the sort method. Hence this is a compile time polymorphism. The output of above program is "Inside Collection sort method"

- **Example for Dynamic Polymorphism:**

```
class Vehicle {
   public void start() {
      System.out.println("Inside start method of Vehicle");
   }
}
class Car extends Vehicle {
   public void start() {
      System.out.println("Inside start method of Car");
   }
}
public class DynamicBindingTest {
        public static void main(String args[]) {
             Vehicle vehicle = new Car(); //here Type is vehicle but object will be Car
            vehicle.start();      //Car's start called because start() is overridden method
        }
}
```

In the above example the super class Vehicle's reference is referring to the object of sub class Car. When we invoke the start method, at compile time the compiler will check whether

the method is present in the class of the reference variable ie Vehicle class. But this doesn't bind the method call to the actual method definition as the Super Class reference type can refer to Sub class object.

At the time of execution, the JVM will check to which object the reference is referring to and accordingly it will invoke the respective method version. In the above example, the Vehicle reference is referring to Car object and hence the start() method defined in Car class will be invoked. This concept of binding the method definition at run time based on the object to which the reference variable is referring to is called Dynamic Method dispatch. The output of above program is "Inside start method of Car"

V. Upcasting and downcasting:

- Upcasting is changing type by moving up the inheritance hierarchy. This is always safe, because it's always true that a SubType IS-A SuperType (for example, it's always true that a ContractEmployee IS A Employee). Therefore, an explicit cast is NOT needed when upcasting.

- Downcasting is changing type by moving down the inheritance hierarchy. This is NOT always safe, because it's NOT always true that a SuperType IS-A SubType (for example, it's NOT always true that a Employee IS A ContractEmployee). Therefore, an explicit cast IS required when downcasting.

- As you know, all objects have a type. The thing to keep in mind is the true runtime type of the object. This is the type of object that was created with "new." Once an object is created, its true runtime type never changes.

But references (stored in variables) also have types. A variable referencing an object can be the same type as that object, or it can be any supertype of that object. For example, a variable that is type "Car" can reference an object that is simply a Car. Or it can reference an object that is a more specific type (a subtype) of Car. For example, a Car variable might reference an instance of "Volkswagen," which IS-A Car.

- The compiler only knows the types of references. It doesn't know the true runtime type of an object. So when you say...

SuperType sup = new SubType();

...that's fine. Because "new SubType()" returns a reference of type SubType. The compiler knows it's always true that a SubType IS-A SuperType (a Volkswagen is always a Car). So it's okay to assign that reference to a variable of type SuperType. (This is called an "assignment conversion.")

On the other hand, a Car is NOT always a Volkswagen.

SubType sub = new SuperType();

...will fail to compile, because "new SuperType()" returns a reference of type SuperType, and the compiler knows that a SuperType is NOT a Subtype.

But suppose the SuperType reference on the right side did, in fact, point to an instance of SubType. For example...

SubType sub = sup; //using the sup we assigned above

To assure the compiler that this assignment is valid, the programmer must insert an explicit cast to downcast the SuperType reference (sup) to type SubType...

SubType sub = (SubType)sup;

- With the explicit cast, the compiler will trust the programmer in saying that the object referenced by sup is, in fact, a SubType object. In this example, there is no problem at runtime, because the true runtime type of the object referenced by "sup" is SubType.

But if it turns out that the downcast is not correct, then the program will fail at runtime with a ClassCastException. This is what's happening with...

SubType sub = (SubType) new SuperType();

"new SuperType()" creates an object with a true runtime type of SuperType, and returns a reference that also has the type SuperType. However, an explicit cast tells the compiler to downcast that reference to SubType, so it compiles. However, at runtime, the program fails because the true type of the object is NOT SubType.

VI. **@override Annotation:**

- @Override annotation informs the compiler that the element is meant to override an element declared in a superclass.

```
// mark method as a superclass method that has been overridden
@Override
int overriddenMethod() { }
```

- While it is not required to use this annotation when overriding a method, it helps to prevent errors. If a method marked with @Override , when misspelling a method name or not giving the correct order of methods as it is in superclass , the compiler generates an error.

4.1.3. Pros & Cons

Pros:

Objects belonging to different types to respond to method, field, or property calls of the same name, each one according to an appropriate type-specific behavior.

Cons:

Overloaded methods should be used sparingly, as they can make code much less readable.

4.1.4. Syntax

4.2. Code Snippets

4.3. Real Time Usage

The primary usage of polymorphism in object-oriented programming is the ability of objects belonging to different types to respond to method, field, or property calls of the same name, each one according to an appropriate type-specific behavior. The programmer (and the program) does not have to know the exact type of the object in advance, and so the exact behavior is determined at run-time (this is called late binding or dynamic binding).

The different objects involved only need to present a compatible interface to the clients' (calling routines). That is, there must be public or internal methods, fields, events, and properties with the same name and the same parameter sets in all the superclasses, subclasses and interfaces. In principle, the object types may be unrelated, but since they share a common interface, they are often implemented as subclasses of the same superclass. Though it is not required, it is understood that the different methods will also produce similar results (for example, returning values of the same type).

4.4. Check Yourself

1) Static Polymorphism can be achieved at _____ time.

Ans. Compilation

2) Dynamic Polymorphism can be achieved at _____ time.

Ans. Execution/Run

3) Static Polymorphism can be achieved using:

a)Overloaded methods b)Overridden static methods

c)Overridden final methods d)Overridden instance methods

Ans. a), b), c)

4) Dynamic Polymorphism can be achieved using:

a)Overloaded methods b)Overridden static methods

c)Overridden final methods d)Overridden instance methods

Ans. d)

5) Constructors can be overridden TRUE/FALSE

Ans. FALSE

5. Error Zone - commonly made mistakes

Polymorphism is not the same as method overloading or method overriding. Polymorphism is only concerned with the application of specific implementations to an interface or a more generic base class. Method overloading refers to methods that have the same name but different signatures inside the same class. Method overriding is where a subclass replaces the implementation of one or more of its parent's methods. Neither method overloading nor method overriding are by themselves implementations of polymorphism, they are only the means by which we can achieve polymorphism.

6. Sample code

7. Additional Information (Optional)

8. Best Practices

1) Overloaded methods should be used sparingly, as they can make code much less readable.

9. Summary or Take Away Points

1) Method overriding is the process where a method defined in super class is redefined in sub class. The signature of the method in subclass should be same as the signature of the method in super class.
2) Method overloading is the concept of having different methods having same name but different parameters. The parameters can differ in:
    - Number of parameters
    - Type of parameters
    - Order of parameters
3) Depending on the time (Compilation or Execution of a program) at which the resolution of a method call will be decided, Polymorphism can be classified as :
    - Static Polymorphism and
    - Dynamic Polymorphism

4) Upcasting is changing type by moving up the inheritance hierarchy. This is always safe, because it's always true that a SubType IS-A SuperType . Therefore, an explicit cast is NOT needed when upcasting.
5) Downcasting is changing type by moving down the inheritance hierarchy. This is NOT always safe, because it's NOT always true that a SuperType IS-A SubType. Therefore, an explicit cast IS required when downcasting.
6) @Override annotation informs the compiler that the element is meant to override an element declared in a superclass.

10. Related Topics:

Class relationships, Inheritance, Interface, Abstract class and Final class

11. References

http://docs.oracle.com/javase/tutorial/java/IandI/summaryinherit.html